EE2211 Solver User Guide

How to enter data, choose a calculator, and interpret its results

Use this guide to work through matrix, regression, classification, decision tree, and clustering calculations. It covers every browser tab and all 15 terminal menu tools, with inputs you can paste and results you can check.

Updated 13 October 2026

Start the program

Install Python 3.11 or newer. Open a terminal in the solver folder, install the app, and start the browser interface:

```
python -m pip install -e .
python -m streamlit run webui.py
```

Open the Local URL printed by Streamlit, usually http://localhost:8501. Keep the terminal open while using the app. For the terminal menu, run `python main.py`. If you use uv, run `uv sync` and `uv run streamlit run webui.py`.

Find the right tool

Browser tab	What it does	Guide page
Array	Inspect matrices and available inverses	2
Solve	Solve Xw = y or wX = y	2
Regression	Linear, polynomial, ridge, and classification	3 to 5
Pearson	Correlation between columns	6
Gradient	Symbolic gradient descent	6
Decision Tree	Classification impurity or regression splits	7 and 8
K-means	Cluster points using initial centroids	9
Cache	Copy and manage recent matrices	9

The terminal menu map and troubleshooting advice are on page 10. Change any settings, enter the data, then press the main action button: Inspect, Solve, Fit, Calculate, or Run.

Matrix inputs and linear systems

Enter rows and columns

In browser boxes, use one sample per row and one feature per column. Separate values with spaces or commas; separate rows with new lines or semicolons. All rows must have equal widths. Numeric inputs must be finite; empty arrays, NaN, infinity, and three-dimensional arrays are rejected.

```
1 2; 3 4
```

```
[[1, 2], [3, 4]]
```

Both examples mean a 2 by 2 matrix. For a column of values, use `0;1;2;3`. Typing `0 1 2 3` creates a row; Transpose changes it into a column. A Python list such as `[0,1,2,3]` is interpreted as a column. Use a nested list `[[0,1,2,3]]` for an explicit row.

Each browser matrix box has a Preview and a Transpose button. Transpose changes the input; press the tool's main action button afterwards. The terminal accepts one line at a time, so use semicolons or a one-line Python array.

Array inspection

Open Array, enter a matrix, and press Inspect. The result shows its dimensions, rank, determinant if square, and whether ordinary, left, and right inverses exist. A left inverse requires independent columns; a right inverse requires independent rows.

```
Array: 1 2; 3 4
```

Expected rank: 2. Determinant: -2. Inverse: `[[-2,1],[1.5,-0.5]]`. For a rank-deficient matrix, the app explains which inverses do not exist.

Solve Xw equals y

Open Solve and select Xw = y. X and y must have the same number of rows. Each column of y is a separate right-hand side. Enter the example below and press Solve.

```
X: 1 1; 1 2; 1 3
y: 2; 3; 4
```

Expected w: `[[1],[1]]`, giving y exactly. The app distinguishes a unique exact solution, many exact solutions, and a least-squares fit when no exact solution exists. For many solutions, it returns the one with minimum Euclidean norm.

Solve wX equals y

Choose wX = y when the unknown multiplies X on the left. X and y must have the same number of columns. Enter X as `1 2 3; 0 1 0` and y as `2 7 6`. The answer is w = `[[2,3]]`.

The residual is fitted y minus entered y. MSE is the average squared residual per target. The displayed standard error is the square root of MSE, also called RMSE; it is not an estimate of uncertainty in the coefficients.

Linear and polynomial regression

Regression uses X for training features and Y for matching targets. Use one sample per row and keep feature columns in the same order in X_test. Multiple numeric target columns can be fitted together.

Linear regression

In Regression, select Model = Linear, Y interpretation = Numeric regression, and Regularization = None. Keep Add bias for linear model checked if the model needs an intercept. Enter the following data and press Fit:

```
X: 0;1;2;3
Y: 1;3;5;7
```

The fitted model is y = 1 + 2x. Weights are `[[1],[2]]`, with the intercept first. Training MSE is approximately zero. With bias enabled, enter raw features rather than a manual column of ones.

Enable Predict X_test to make predictions. Enter X_test = `4;5`; predictions are `9;11`. Optionally enter Y_test = `9;12` to obtain test SSE = 1 and test MSE = 0.5. Leave Y_test empty if you only need predictions.

Polynomial regression

Choose Model = Polynomial, set Polynomial order = 2, keep Numeric regression and Regularization = None, and enter:

```
X: 0;1;2
Y: 1;4;9
X_test: 3
```

Press Fit with Predict X_test enabled. The model is y = 1 + 2x + x squared, with weights `[[1],[2],[1]]`. The prediction at x = 3 is 16.

Polynomial expansion always includes the constant term. Do not add a bias column yourself. For two input features and order 2, columns of the model matrix P are 1, x1, x2, x1 squared, x1 times x2, and x2 squared. The app displays P so you can match each weight to its feature.

Read the outputs

Model Matrix is the data actually used for fitting. Weights contains the fitted coefficient vector or matrix. Training Prediction gives the fitted targets. Square error is SSE, the sum of squared errors; MSE divides each target column's SSE by the number of samples. Test errors use only the entered test samples.

Dependent feature columns are supported through a minimum-norm least-squares fit. A zero training error alone does not establish that the model will predict new data well. Choose an appropriate polynomial degree for the problem.

Terminal equivalents

Use menu 4 for linear regression or menu 5 for polynomial regression. Follow the prompts for X, Y, bias or order, then optionally X_test and Y_test. A blank optional input finishes the calculation.

Ridge regression

Ridge penalizes large coefficients using Lambda. It works with either a linear or polynomial model, including square model matrices. Lambda must be finite and nonnegative. The implementation penalizes every model coefficient, including the constant term.

Ridge linear regression

In Regression, select Linear, Numeric regression, and Regularization = Ridge. Keep Add bias checked, set Lambda = 1, and select Ridge form = auto. Enter:

```
X: 0;1
Y: 1;3
X_test: 2
```

Enable Predict X_test and press Fit. The two weights are `[[1],[1]]`. Predictions on the training samples are 1 and 2, so training SSE = 1 and MSE = 0.5. The test prediction at x = 2 is 3.

These weights differ from the unregularized solution `[[1],[2]]` because Lambda applies even when the model matrix is square.

Ridge polynomial regression

Choose Polynomial, order 2, Numeric regression, and Ridge. Set Lambda = 1 and auto, then enter:

```
X: 0;1
Y: 1;3
X_test: 2
```

Press Fit with prediction enabled. Polynomial columns are 1, x, and x squared. The weights are approximately `[[0.8571429],[0.7142857],[0.7142857]]`; the prediction at x = 2 is approximately 5.1428571.

Choose a form

Ridge form	Use
auto	Primal when samples >= model columns; dual when samples < columns
primal form	Solve the regularized system over model columns
dual form	Solve the regularized system over sample rows

Primal and dual forms produce the same ridge solution up to numerical rounding. Lambda = 0 gives the ordinary minimum-norm least-squares solution, including when columns are dependent. Increasing Lambda usually shrinks weights and can increase training error.

Terminal equivalents

Menu 6 runs ridge linear regression and always adds a bias term. Menu 7 runs ridge polynomial regression, which includes a constant automatically. Enter X, Y, Lambda, and form; menu 7 also asks for polynomial order. Then enter optional test features and targets.

Classification with regression models

The Regression tab also fits classification scores. Select Linear or Polynomial and choose a Y interpretation. These tools use least-squares or ridge fitting; their raw scores can fall outside the range from zero to one.

One hot linear classification

Choose Linear, One-hot classification, Regularization = None, and Add bias checked. Enable Predict X_test and enter:

```
X: 0;1;2;3
Y: low;low;high;high
X_test: 2.5
Y_test: high
```

Press Fit. Classes are stored in order of first appearance: low, then high. The test scores are approximately `[[0.1,0.9]]`, so the predicted label is high. The largest score chooses the class. A tie chooses the first class column.

You may use a single column of text or numeric labels. Each unique label becomes a separate class. Alternatively enter one-hot rows such as `1 0;1 0;0 1;0 1`. Each row must contain exactly one 1, with all other entries 0.

One hot polynomial classification

Select Polynomial and order 2, then use the same training and test data. The polynomial model includes a constant automatically. It predicts high at 2.5. Menu 8 runs one-hot linear classification; menu 9 runs one-hot polynomial classification. The terminal prints scores and classified labels when labels were supplied.

Sign classification

For binary labels -1 and +1, choose Sign classification in the browser. With Linear, Add bias unchecked, no regularization, X = `-1;1`, Y = `-1;1`, and X_test = `-2;0;2`, predictions and signs are `-2;0;2` and `-1;0;1`. A score of exactly zero is shown as class 0.

Choose Auto deliberately

Y data	Auto interpretation
Text labels	One-hot classification
One numeric column containing only -1 and +1	Sign classification
Valid one-hot rows with multiple columns	One-hot classification
Other numeric targets	Numeric regression

Select Numeric regression explicitly if your multi-column numeric targets happen to look one-hot. Select One-hot classification explicitly for numeric category labels such as 0, 1, and 2.

For label-based training, Y_test must use known training labels. For one-hot training, use one-hot test rows in the same column order, or a single column of zero-based class indices. Test MSE compares score vectors with one-hot targets; it is not classification accuracy.

Pearson correlation and gradient descent

Pearson correlation

Open Pearson. Enter X and Y with observations in rows and variables in columns, then press Calculate. The output includes means, population standard deviations and variances, covariance, and Pearson correlation for each X-column and Y-column pair.

```
X: 1;2;3;4
Y: 2;4;6;8
```

Expected correlation: 1. X has mean 2.5 and variance 1.25; Y has mean 5 and variance 5. Covariance is 2.5. A value near -1 means a negative linear relationship; near +1 means a positive linear relationship; near zero means little linear association.

Both inputs need matching observation counts and at least two observations. Constant columns have undefined correlation and are rejected. Very small but nonconstant values are accepted. Matching single-row inputs are automatically transposed; certain matching-column layouts are also transposed. Prefer observation rows to make the intended orientation clear. Menu 10 runs the same tool.

Gradient descent

Open Gradient. Enter a cost function with Python or SymPy notation: use `**` for powers, `*` for multiplication, and radians for trigonometric functions. The displayed Variable order is alphabetical; enter one initial value per variable in that order, separated by commas.

```
Cost function: x**2 + x*y**2
Initial values: 3, 2
Learning rate: 0.2
Iterations: 1
Tolerance: leave empty
```

Press Run. At the initial point, the gradient is `[10,12]`. One simultaneous update gives x = 1 and y = -0.4. The gradient at that new point is `[2.16,-0.8]`, and the cost is 1.16.

The history includes iteration 0, followed by each update. Derivatives and costs in a row are evaluated at the parameters in that row. Final Parameters contains the last parameter vector.

Learning rate must be positive and finite. Leave Tolerance empty to run the requested number of updates. With tolerance set, the program stops when the largest absolute parameter change is at most that nonnegative tolerance. Equal consecutive costs alone do not stop an oscillating sequence.

If a function or gradient leaves its real, finite domain, the program reports an error. Check the initial point and reduce the learning rate if needed. Menu 12 asks for the same function, initial values, learning rate, iterations, and optional tolerance.

Classification decision tree impurity

Open Decision Tree and choose Classification impurity. Each row contains class counts in the same class order. By default, First row is the parent node is checked: enter the parent first, followed by its disjoint children. The parent is used for comparison and gain; only the children contribute to the weighted result.

Calculate the example with nodes A B and C

```
8 5 5
6 4 0
2 1 5
```

A is the parent, B is the first child, and C is the second child. Keep the parent option checked, select gini or all, and press Calculate. The parent row must equal the column-wise sum of its children; here A = B + C.

Node	Samples	Weight	Gini	Contribution
B child	10	10/18	0.48	0.2666667
C child	8	8/18	0.53125	0.2361111

Weighted Gini is 0.5027778. Gini gain is parent impurity minus weighted child impurity: 0.1453704. The parent samples are already represented in B and C, so including A in the weighted average would count them twice.

Choose an impurity measure

Measure	Parent	Weighted children	Gain
gini	0.6481481	0.5027778	0.1453704
entropy	1.5466316	1.1166592	0.4299724
misclassification	0.5555556	0.3888889	0.1666667

For a node, divide each class count by its sample total to get class probabilities. Gini is one minus the sum of squared probabilities. Entropy uses base-2 logarithms, with zero-probability terms contributing zero. Misclassification impurity is one minus the largest class probability. Select all to see all three measures.

Enter children only

Uncheck First row is the parent node and enter `6 4 0;2 1 5` to get the same weighted result. The app reconstructs the parent by summing children. Each child weight is its size divided by the sum of child sizes. A zero-sample branch has weight and contribution zero; at least one branch must contain samples.

Use disjoint nodes from a single split or a set of disjoint leaves. Do not mix several ancestor levels into the same calculation. Menu 13 asks whether the first row is the parent. The default answer is yes.

Course reference: ee2211book.pdf, Chapter 9, pages 62 to 66; weighting and gain are illustrated in equations 9.6 and 9.7 on page 64.

Regression decision tree splits

Open Decision Tree and choose Regression MSE / split. Enter one feature x and one target y for every sample. The calculator evaluates a single split; it does not grow a complete tree. Feature and target vectors must have matching lengths and finite values.

Check a threshold

```
x values: 1;2;3
y values: 1;5;9
Decision threshold: 2
```

Press Calculate. The split convention matches the course book: the left child contains x values strictly less than the threshold; the right contains x values greater than or equal to it. Thus x = 2 belongs to the right child.

Quantity	Expected result
Root MSE	10.6666667
Left mean and MSE	1 and 0
Right mean and MSE	7 and 4
Weighted split MSE	(1/3) times 0 + (2/3) times 4 = 2.6666667

A node predicts the mean of its training targets. Its MSE is the average squared distance from that mean. Overall split MSE weights each child's MSE by its number of samples divided by the parent sample count.

Find the best one level split

Enable Find best one-level split. You can leave Decision threshold empty to search automatically. The calculator keeps x and y paired while sorting x, evaluates thresholds between distinct x values, and chooses the lowest weighted MSE. Repeated feature values stay together. If candidates tie, the smallest candidate threshold is shown.

For the small example above, thresholds 1.5 and 2.5 have equal MSE, so the selected best threshold is 1.5. A manually entered threshold must put at least one sample on each side. Automatic search needs at least two distinct x values.

Reproduce the book house price example

```
x values: 0.5;0.6;1;2;3;3.2;3.8
y values: 0.19;0.23;0.28;0.42;0.53;0.75;0.80
```

Enable the search and press Calculate. The best threshold is 2.5. Weighted split MSE is approximately 0.0102095, which rounds to the book's 0.0102. To compare several features, rerun with each feature's x values while keeping targets aligned with their samples.

Menu 14 accepts the same vectors and optional threshold, then asks whether to search for the best split. Cache entries named threshold_summary contain threshold, root MSE, left MSE, right MSE, and split MSE in that order. best_split contains threshold followed by split MSE.

Course reference: ee2211book.pdf, Chapter 9, pages 67 and 68; equations 9.17 to 9.19 define node MSE and sample-weighted split MSE.

K means clustering and cached results

K means clustering

Open K-means. Enter Points X with one point per row and Initial centroids with one centroid per row. The number of centroid rows determines k. Both matrices must have the same number of feature columns.

```
Points X: 1 1;1.5 2;5 7;6 8
Initial centroids: 1 1;5 7
Max iterations: leave empty
```

Press Run. The point assignments are clusters 1, 1, 2, and 2. Final centroids are `[[1.25,1.5],[5.5,7.5]]`. The initial cost is 3.25 and the final cost is 1.625.

Points are assigned to the nearest centroid by Euclidean distance. Each centroid is then replaced by the mean of its assigned points. Cost is the sum of squared distances from points to their assigned centroids. Cluster numbers start at 1 in the interface; a distance tie chooses the first centroid.

An empty cluster keeps its previous centroid. Leave Max iterations empty to run until convergence, or enter a positive integer to cap updates. With Max iterations = 1, the example computes the updated centroids but may stop before confirming convergence. The initial assignment is iteration 0. Menu 15 runs the same clustering workflow.

Copy and reuse results

Open Cache to see recent inputs, weights, predictions, tree calculations, and centroids. Entries show their source, dimensions, and time. New entries appear first. Use Copy to paste an array into another tool; use Remove for one item or Clear cache for all cached matrices.

Max allowed cache controls how many matrices are retained; the default is 50. Reducing the limit removes older entries. Displayed numbers are rounded for readability, while copied arrays and transposed inputs retain their stored floating-point precision.

The browser saves widget inputs and cache data under `temp/webui_state.json`, so local sessions can restore them. Cached matrices are working data, not a fitted model object. To predict with the interface, enter the original training data and optional X_test in Regression, or copy a coefficient matrix into Solve when appropriate.

Terminal cache

Menu 11 shows the last numeric input and last result. At a numeric matrix prompt, a single `-` reuses the last input and _ reuses the last result. A minus sign within a number still denotes a negative value. The terminal cache lasts only for the current run.

If Copy is unavailable in your browser, use the displayed Python array text in the cache entry. Keep the complete array when transferring a result so rounding does not alter later calculations.

Terminal tools and troubleshooting

Terminal menu map

Menu	Tool	Shortcut	Page
1	Array inspection	inspect	2
2	Solve Xw = y	solve	2
3	Solve wX = y	left solve	2
4	Linear regression	linear	3
5	Polynomial regression	poly	3
6	Ridge regression	ridge	4
7	Ridge polynomial regression	ridgepoly	4
8	One-hot linear classification	onehot	5
9	One-hot polynomial classification	onehotpoly	5
10	Pearson correlation	pearson	6
11	Show cached input and result	cache	9
12	Gradient descent	gradient	6
13	Classification tree impurity	impurity	7
14	Regression tree MSE and split	split	8
15	K-means clustering	kmeans	9

Press Enter to accept bracketed defaults or skip optional inputs. Enter 0, exit, or q at the main menu to exit. Ctrl+C during a calculator returns to the menu.

Resolve common input problems

Sample count mismatch: X and Y need the same number of sample rows. Use semicolons for a column or press Transpose. **Feature mismatch:** X_test must have the same feature columns as raw X. **Parent class count mismatch:** ensure each parent class count equals the sum of that class in the children, or select children-only input.

Unknown test class: use a training label, or a valid zero-based column index when training targets were one-hot. **Undefined Pearson correlation:** remove or change constant columns. **Empty tree split:** choose a threshold that separates distinct feature values. **Nonfinite gradient:** check the function domain and learning rate.

Verify the installation and calculations

```
python run_tests.py
python -m unittest test_webui -v
```

The first command checks numerical calculations. The second exercises the browser forms with isolated test state. See README.md for alternative Python commands and virtual environment setup.